Вариант 1: Обработка файлов в многопоточном режиме

Разработать многопоточное приложение, которое параллельно считывает несколько файлов и выполняет их обработку.

Каждый файл должен обрабатываться в отдельном потоке, при этом необходимо обеспечить корректную синхронизацию между потоками и исключить возможность гонки данных.

После завершения обработки всех файлов программа должна вывести итоговую статистику, включая количество обработанных файлов и общее время выполнения.

Требования

1. Параллельная обработка файлов
   - Использовать Boost.Thread для создания потоков, каждый из которых обрабатывает один файл.
   - Потоки должны работать независимо и параллельно.
2. Синхронизация доступа к общим ресурсам
   - Использовать std::mutex для защиты общих данных (если требуется).
   - Гарантировать отсутствие гонок данных при обновлении общего счетчика обработанных файлов.
3. Контроль выполнения потоков
   - Использовать std::condition_variable для синхронизации потоков и ожидания завершения работы.
   - Основной поток должен ожидать завершения всех потоков перед завершением программы.
4. Безопасное обновление счетчика обработанных файлов
   - Использовать std::atomic<int> вместо std::mutex для хранения количества обработанных файлов.
5. Измерение времени выполнения обработки файлов
   - Использовать std::chrono для измерения общего времени обработки всех файлов.

Пример работы программы

1. Допустим, у нас есть три файла

data1.txt

data2.txt

data3.txt

2. Программа запускает отдельные потоки для обработки этих файлов.
3. Каждый поток выполняет следующие операции
   - Открывает файл и читает его содержимое.
   - Производит обработку (например, подсчет количества слов и символов).
   - Записывает результаты в выходной файл (processed_data1.txt, processed_data2.txt, ...).
   - Уведомляет основной поток о завершении работы.
4. Основной поток ждет завершения всех потоков.
5. В конце программа выводит
   - Количество обработанных файлов.
   - Общее время обработки.

Дополнительные задания

1. Реализовать очередь задач
   - Вместо фиксированного списка файлов использовать очередь задач (task queue).
   - Потоки должны получать файлы для обработки из этой очереди.
2. Использовать пул потоков
   - Вместо создания одного потока на каждый файл организовать фиксированный пул потоков (например, 2 потока).
   - Каждый поток берет файл из очереди и обрабатывает его.
3. Обрабатывать файлы по частям
   - Разделить файл на блоки и обрабатывать их в нескольких потоках одновременно.
4. Считать статистику по файлам
   - Подсчитать количество слов и символов в каждом файле.
   - Вывести результаты в консоль.
5. Записывать обработанные данные в новый файл
   - Создавать выходной файл processed_<имя_файла>.txt для хранения результатов обработки.

Вариант 2: Многопоточное вычисление суммы элементов матрицы

Разработать многопоточное приложение, которое вычисляет сумму всех элементов большой матрицы.

Для эффективного выполнения необходимо разделить матрицу на части и параллельно обрабатывать их в нескольких потоках, обеспечивая синхронизацию доступа к общей переменной суммы.

После завершения вычислений программа должна вывести итоговую сумму и время выполнения.

Требования

1. Разделение матрицы на части
   - Разделить матрицу на фрагменты, каждый из которых обрабатывается в отдельном потоке.
   - Возможные стратегии разделения:
   - По строкам (каждый поток обрабатывает свою часть строк).
   - По блокам (разделение на квадратные фрагменты размером N x N).
2. Параллельное вычисление суммы элементов
   - Использовать Boost.Thread для создания потоков, работающих параллельно.
3. Синхронизация доступа к общей переменной суммы
   - Использовать std::mutex для предотвращения гонки данных при обновлении глобальной переменной суммы.
4. Контроль выполнения потоков
   - Использовать std::condition_variable, чтобы основной поток ждал завершения всех рабочих потоков перед выводом результата.
5. Измерение времени выполнения
   - Применить std::chrono для измерения общего времени вычислений.

Пример работы программы

1. Исходная матрица 1000×1000 заполняется случайными числами.
2. Программа запускает 4 потока, которые обрабатывают свои части матрицы.
3. Каждый поток выполняет
   - Суммирует элементы своей части матрицы.
   - Обновляет общий результат.
   - Уведомляет основной поток о завершении работы.
4. Основной поток ждет завершения всех потоков.

5. В конце программа выводит
   - Итоговую сумму элементов матрицы.
   - Общее время выполнения.

Дополнительные задания

1. Разделение матрицы на блоки
   - Вместо обработки строк потоки работают с блоками (N x N).
   - Например, 1000×1000 можно разделить на блоки 250×250, каждый из которых обрабатывается отдельно.
2. Многопоточное сложение двух матриц
   - Вместо одной матрицы использовать две (A + B → C).
   - Каждый поток обрабатывает соответствующую часть A и B, записывая сумму в C.
3. Использование std::future для асинхронного сбора результатов
   - Вместо явной синхронизации потоков использовать std::async + std::future для получения промежуточных сумм.
4. Подсчет обработанных блоков с std::atomic<int>
   - Добавить счетчик обработанных блоков, который потоки обновляют с использованием std::atomic<int>.
   - После обработки всех блоков программа выводит статистику.
5. Запись результата в файл
   - Если выполняется сложение двух матриц (A + B), записать итоговую C в result_matrix.txt.

Вариант 3: Многопоточная обработка очереди задач

Разработать многопоточное приложение, реализующее обработчик задач (task queue), который распределяет вычислительные задачи между несколькими потоками.

Рабочие потоки извлекают задачи из общей очереди и выполняют их, а основной поток координирует работу и ожидает завершения всех потоков.

Задачи

1. Вычисление факториала (N!)
2. Поиск N-го числа Фибоначчи
3. Подсчёт суммы цифр в числе N
4. Проверка, является ли число N простым
5. Вычисление наибольшего общего делителя (НОД) двух чисел
6. Обратный порядок цифр в числе (реверс числа)

Программа должна обеспечивать корректную синхронизацию при доступе к очереди задач и гарантировать завершение всех потоков после выполнения всех заданий.

Требования

1. Очередь задач (task queue)
   - Задачи помещаются в общую очередь задач.
   - Потоки извлекают задачи из очереди и выполняют их.
2. Рабочие потоки
   - Использовать Boost.Thread для создания нескольких рабочих потоков, которые выполняют задачи параллельно.
   - Потоки завершают работу после выполнения всех задач.
3. Синхронизация потоков
   - Использовать std::mutex для безопасного доступа к очереди задач.
   - Применить std::condition_variable для координации работы потоков и ожидания появления новых задач.
4. Гарантированное завершение всех потоков
   - Потоки завершаются корректно после выполнения всех задач.
   - Основной поток ожидает завершения работы всех потоков перед завершением программы.
5. Измерение времени выполнения
   - Использовать std::chrono для замера общего времени обработки всех задач.

Пример работы программы

1. Есть 6 задач в очереди
   - Вычисление факториала числа N!.
   - Поиск N-го числа Фибоначчи.
   - Подсчет суммы цифр в числе N.
   - Проверка, является ли число N простым.
   - Вычисление наибольшего общего делителя (НОД) двух чисел.
   - Реверс числа (обратный порядок цифр).
2. 3 рабочих потока распределяют задачи между собой.
3. Каждый поток
   - Извлекает задачу из очереди.
   - Выполняет вычисления.
   - Сообщает о завершении задачи.
4. Основной поток ожидает завершения всех потоков.
5. Программа выводит:
   - Результаты всех задач.
   - Общее время обработки всех задач.

Дополнительные задания

1. Многопоточное выполнение разных типов задач
   - Потоки получают случайные задачи из набора (факториал, числа Фибоначчи, простые числа и т. д.).
2. Реализация пула потоков
   - Создать фиксированное число потоков, динамически распределяющих задачи.
3. Очередь задач с приоритетами
   - Реализовать очередь с приоритетами, где более сложные задачи выполняются в первую очередь.
4. Использование std::future для асинхронного получения результатов
   - Рабочие потоки возвращают результаты вычислений через std::future.
5. Добавить механизм остановки потоков
   - После выполнения всех задач потоки корректно завершаются.

Вариант 4: Многопоточная симуляция банка

Разработать многопоточное приложение, моделирующее обработку банковских транзакций несколькими потоками.

В симуляции несколько клиентов совершают случайные переводы средств между своими счетами, а потоки управляют этими операциями, обеспечивая синхронизацию и защиту от гонок данных.

После выполнения всех транзакций программа выводит итоговые балансы клиентов, а также измеряет общее время обработки.

Требования

1. Обработка транзакций в нескольких потоках
   - Использовать Boost.Thread для обработки банковских операций.
   - Несколько потоков одновременно совершают переводы средств между клиентами.
2. Синхронизация доступа к счетам
   - Использовать std::mutex для предотвращения состояния гонки при обновлении балансов.
   - Применить std::condition_variable для координации работы потоков и ожидания доступа к счетам.
3. Гарантированная корректность обновления балансов
   - Исключить состояния гонки при одновременном доступе к счетам.
   - Обеспечить атомарность операций при изменении общего баланса банка.
4. Измерение времени выполнения
   - Использовать std::chrono для замера времени обработки транзакций.

Пример работы программы

1. Исходные данные
   - 10 клиентов, у каждого из которых есть начальный баланс.
   - 5 потоков, обрабатывающих 100 случайных транзакций.
2. Процесс обработки транзакций
   - Каждый поток случайно выбирает двух клиентов.
   - Совершает перевод случайной суммы от одного клиента к другому.
   - Использует std::mutex для защиты данных и исключения гонки.
3. Выходные данные

- После завершения всех транзакций программа выводит итоговые балансы клиентов.
- Вычисляется и отображается общее время выполнения.

Дополнительные задания

1. Реализация структуры клиентских счетов
   - Каждый клиент имеет уникальный баланс, который обновляется при переводах.
2. Многопоточное обновление счетов с защитой std::mutex
   - При переводе средств между клиентами используются блокировки мьютексов для защиты данных.
3. Использование std::atomic<double> для подсчёта общего баланса
   - Общий баланс банка рассчитывается атомарно, чтобы избежать несогласованности данных.
4. Запись многопоточного лога транзакций в файл (transactions.log)
   - Каждая транзакция записывается в файл с указанием времени и сумм перевода.
5. Остановка после N транзакций
   - Программа завершается автоматически, когда выполняется заданное число переводов.